Apuntes · Sesión 3 · Combinar promesas

Nov 5, 2026 · @Marce

Hasta ahora has encadenado pasos que dependían unos de otros. En esta sesión aprendes a lanzar varias operaciones **a la vez** y a combinar sus resultados, y a gestionar los errores con más control.

En los ejemplos se usa esta función, que simula una operación que tarda `ms` milisegundos y, si `falla` es `true`, falla:

```
const tarea = (nombre, ms, falla = false) => new Promise((resolve, reject) =>
{
  setTimeout(() => {
    if (falla) reject(new Error(nombre + ' ha fallado'));
    else resolve(nombre);
  }, ms);
});
```

`falla = false` es un **valor por defecto**: si no pasas el tercer argumento, vale `false`.

1. En serie o en paralelo

Si unas operaciones **no dependen** unas de otras, no hace falta esperar a que termine una para empezar la siguiente.

En serie: cada tarea empieza cuando termina la anterior.

```
tarea('A', 1000)
  .then(() => tarea('B', 1000))
  .then(() => tarea('C', 1000))
  .then(() => console.log('Terminadas'));   // al cabo de 3 segundos
```

En paralelo: las tres empiezan a la vez.

```
Promise.all([tarea('A', 1000), tarea('B', 1000), tarea('C', 1000)])
  .then(() => console.log('Terminadas'));   // al cabo de 1 segundo
```

En la segunda versión, al escribir el array ya se llama a las tres funciones, así que los tres temporizadores arrancan en el mismo instante. `Promise.all` solo espera a que terminen todas.

Es como hacer la compra: si vas tú a la panadería, luego a la frutería y luego a la carnicería, tardas la suma de los tres tiempos. Si vais tres personas a la vez, tardáis lo que tarde la más lenta.

Para medir cuánto tarda un trozo de código puedes usar `console.time('nombre')` al empezar y `console.timeEnd('nombre')` al terminar.

> **Ahora te toca: ejercicio 3.1.** En `alumnado/sesion3/ejercicios.js`, lanza tres tareas en serie y en paralelo, y mide cuánto tarda cada versión con `console.time`. Enunciado completo en la hoja de ejercicios.

2. Los cuatro combinadores

Los cuatro reciben un **array de promesas** y devuelven **una sola promesa**. Se diferencian en cuándo se cumple o se rechaza esa promesa.

Promise.all: las necesito todas

```
Promise.all([tarea('Precio', 600), tarea('Fotos', 300), tarea('Opiniones',
900)])
  .then(resultados => console.log(resultados))
  .catch(err => console.log('Error:', err.message));
// al cabo de 900 ms: [ 'Precio', 'Fotos', 'Opiniones' ]
```

- Se cumple cuando se han cumplido **todas**, con un array de resultados **en el mismo orden** que el array de entrada (no en el orden en que terminan).
- Si **una** falla, se rechaza enseguida con ese error y los demás resultados se pierden. Prueba a poner `true` como tercer argumento de `'Fotos'`.

Promise.allSettled: quiero saber cómo fue cada una

```
Promise.allSettled([tarea('Foto 1', 400), tarea('Foto 2', 200, true)])
  .then(resultados => console.log(resultados));
// [ { status: 'fulfilled', value: 'Foto 1' },
//   { status: 'rejected',  reason: Error: Foto 2 ha fallado } ]
```

- Espera a que terminen **todas**, vayan bien o mal, y **nunca se rechaza**.
- Devuelve un informe: para cada promesa, su estado (`status`) y su valor (`value`) o su error (`reason`).

Promise.race: gana la primera que termine

```
Promise.race([tarea('Tortuga', 3000), tarea('Liebre', 1000)])
  .then(ganadora => console.log('Gana:', ganadora));
// al cabo de 1 segundo: Gana: Liebre
```

- Termina igual que la **primera** promesa que termine, tanto si se cumple como si falla.
- Si la más rápida falla, `race` se rechaza con ese error.
- Un uso típico es poner un tiempo máximo a una operación: se hace una carrera entre la operación y un temporizador que falla.

Promise.any: me vale la primera que vaya bien

```
Promise.any([tarea('Proveedor 1', 300, true), tarea('Proveedor 2', 700),
tarea('Proveedor 3', 500)])
  .then(r => console.log('Respuesta de', r));
// al cabo de 500 ms: Respuesta de Proveedor 3
```

- Se cumple con la primera que se **cumpla**, ignorando las que fallan.
- Solo se rechaza si fallan **todas**.

Resumen de los cuatro

Método	Se cumple cuando…	Se rechaza cuando…
`Promise.all`	se cumplen todas	falla una
`Promise.allSettled`	terminan todas, como sea	nunca
`Promise.race`	la primera en terminar se cumple	la primera en terminar falla
`Promise.any`	se cumple una	fallan todas

> **Ahora te toca: ejercicio 3.2.** En `alumnado/sesion3/ejercicios.js`, elige qué combinador usarías en cada una de las cuatro situaciones y explica por qué. Enunciado completo en la hoja de ejercicios.

3. Errores: recuperar y relanzar

Un `catch` no tiene por qué ir al final de la cadena. Si va en medio, decide si el error se **arregla** o si **sigue adelante**.

Recuperar: el catch arregla el error

Si el `catch` **devuelve** un valor normal, la cadena se considera arreglada y continúa por el siguiente `then` con ese valor:

```
tarea('Cargar foto', 500, true)          // falla
  .catch(err => {
    console.log('Uso una foto por defecto');
    return 'foto-por-defecto.png';        // valor de repuesto
  })
  .then(foto => console.log('Muestro', foto));   // Muestro foto-por-
defecto.png
```

Relanzar: el catch deja pasar el error

A veces quieres hacer algo con el error, como apuntarlo en un registro, pero que siga siendo un error. Para eso se vuelve a lanzar con `throw`:

```
tarea('Guardar', 500, true)
  .catch(err => {
    console.log('Lo apunto en el registro');
    throw err;                                  // el error continúa
  })
  .then(() => console.log('Esto NO se ejecuta'))
  .catch(err => console.log('Catch final:', err.message));
```

Dentro de un `catch` …	La cadena sigue por…
`return valor` (o no devolver nada)	el siguiente `then`: el error queda arreglado
`throw err`	el siguiente `catch`: el error continúa

`throw` dentro de un `then` también sirve para provocar un error a propósito: la promesa de ese `then` se rechaza, igual que con `reject`.

Promesa sin catch

```
tarea('Sin red', 300, true);   // nadie gestiona el error
```

En la consola del navegador aparece en rojo *Uncaught (in promise) Error: Sin red ha fallado*. «Uncaught» significa «no capturado»: había un error y nadie lo atendió. En Node, además, el programa se para.

Regla práctica: toda cadena de promesas termina en un `catch`.

4. Juntar datos de varios pasos

Para la tienda necesitas dos técnicas más: usar un dato de un `then` anterior, y lanzar muchas peticiones a partir de un array.

Usar un dato de un then anterior

Cada `then` solo recibe lo que devolvió el anterior. Si en el último paso necesitas un dato de un paso de antes, **guárdalo en una variable declarada fuera de la cadena**, que la ven todos los `then`:

```
let cliente;                          // declarada fuera

tarea('Ana', 500)
  .then(nombre => {
    cliente = nombre;                 // la guardo para después
    return tarea('3 pedidos', 500);
  })
  .then(pedidos => console.log(cliente, 'tiene', pedidos));
// al cabo de 1 segundo: Ana tiene 3 pedidos
```

En el segundo `then`, `nombre` ya no existe (era el parámetro del primero), pero `cliente` sí.

Muchas peticiones a partir de un array

Si tienes un array de datos y necesitas una petición por cada uno, `map` convierte cada dato en su promesa, y `Promise.all` las espera todas:

```
const ids = [1, 2, 3];
const peticiones = ids.map(id => tarea('Ficha ' + id, 1000 - id * 200));
// [promesa, promesa, promesa]

Promise.all(peticiones)
  .then(fichas => {
    ids.forEach((id, i) => console.log(id, '→', fichas[i]));
  });
// 1 → Ficha 1
// 2 → Ficha 2
// 3 → Ficha 3
```

La ficha 3 es la que termina antes, pero sale en su sitio: `Promise.all` **respeta el orden** del array. Por eso `fichas[i]` es siempre la ficha de `ids[i]`, y se pueden emparejar por la posición `i`.

> **Ahora te toca: ejercicio 3.3, la tienda (entregable).** Completa `alumnado/sesion3/tienda/tienda.js` y abre `tienda/index.html` en el navegador para verla. Necesitarás `Promise.all` con `map`, encadenar, una variable de fuera para tener los productos en el último `then`, `catch` y `finally`. Enunciado completo en la hoja de ejercicios.

> **Ampliación: ejercicio 3.4, timeout.** Si terminas la tienda, escribe `conTimeout(promesa, ms)` en `alumnado/sesion3/ejercicios.js`: una carrera (`Promise.race`) entre la promesa y una espera que lanza un error. Después úsala en la tienda: pruébala con 500 ms para ver el error «Tiempo agotado» y déjala en 2000 ms. Enunciado completo en la hoja de ejercicios.

5. Hacia AJAX

En la tienda simulada de esta sesión, `obtenerProductos()` devuelve una promesa que se cumple con los productos. En el próximo tema pedirás los datos a un servidor de verdad con `fetch`, y `fetch` también devuelve una promesa:

```
fetch('https://servidor.com/productos')
  .then(respuesta => respuesta.json())
  .then(productos => console.log(productos))
  .catch(err => console.log('Error:', err.message));
```

Todo lo que has aprendido sirve igual: `then`, `catch`, `finally`, encadenar y `Promise.all`. Solo cambia de dónde vienen los datos.

6. Resumen

Concepto	En una frase
En serie	Cada operación empieza cuando termina la anterior: tarda la suma de todas
En paralelo	Todas empiezan a la vez: tarda lo que la más lenta
Variable de fuera	Guarda un dato de un then para usarlo en otro posterior
map + Promise.all	Una petición por cada elemento del array; los resultados llegan en el mismo orden

Concepto	En una frase
`Promise.all`	Espera a todas; falla si falla una
`Promise.allSettled`	Espera a todas y da un informe; nunca falla
`Promise.race`	Termina como la primera que termine, bien o mal
`Promise.any`	Se queda con la primera que vaya bien; falla solo si fallan todas
Recuperar	Un `catch` que devuelve un valor arregla el error
Relanzar	Un `catch` con `throw err` deja que el error siga
Sin `catch`	*Uncaught (in promise)*: toda cadena termina en un `catch`